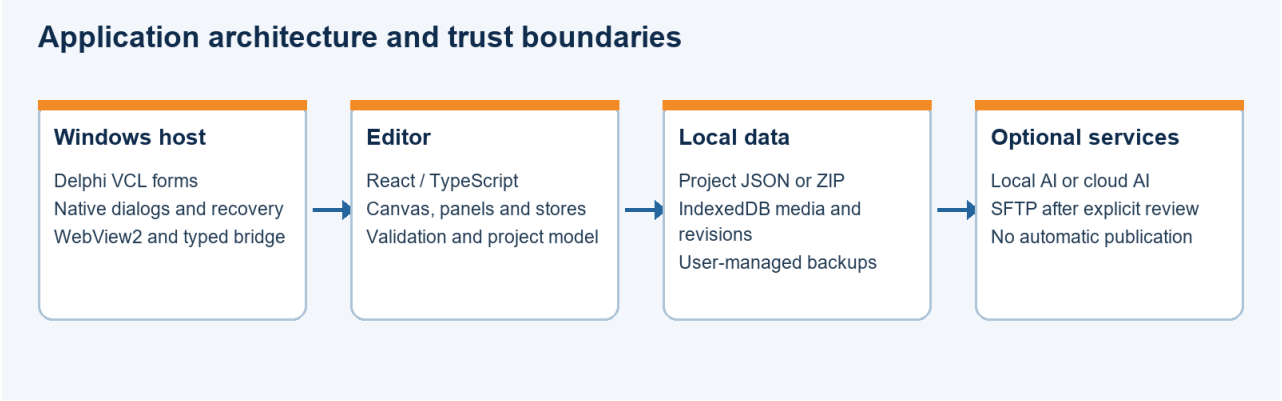
DELPHI WEB STUDIO

Delphi Web Studio Engineering Guide

Architecture, implementation and maintenance

Illustrated edition | Last changed: November 1, 2026

Experimental Windows application



Screens show an isolated example project. Your site content and available options may differ.

Contents

Scope and evidence 3
Source tree and ownership 3
Build tools and configurations 4
Entry, routing and startup 4
Project schema and validation 5
Persistence, media and revisions 6
Native save, backup and recovery 6
Native bridge and trust boundary 7
Rendering, controls and export 7
Static and WordPress import 8
Local AI and optional cloud AI 8
SFTP review, transfer and rollback 9
Validation and known issues 10
Maintenance checklist 11
Main form lifecycle in detail 11
Browser and native request sequence 12
Media record and import mechanics 12
Project file and archive maintenance 12
Testing map and failure interpretation 13
Delivery, architecture and platform limitations 13
Source declaration index 14

Scope and evidence

This guide describes the experimental Delphi Web Studio source and local builds inspected on November 1, 2026. It is an engineering reference, not a claim that the complete handoff or release acceptance has passed. The principal sources are desktop/DelphiWebStudio, desktop/bridge, desktop/project, desktop/ai, src, package.json, tests and the dated validation records. Earlier architecture notes describe the upstream baseline and may be obsolete for current behavior.

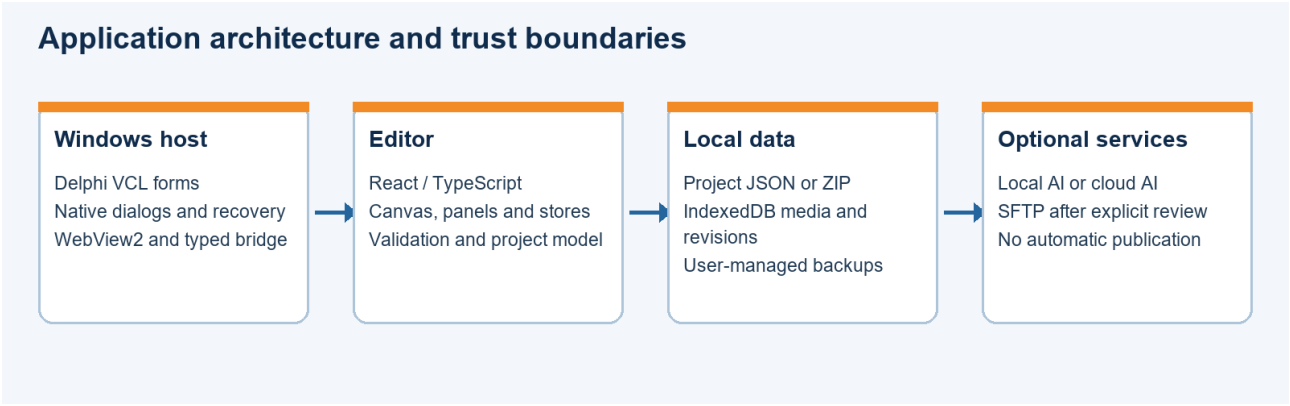


Figure 1. Simplified components and optional services; network use is user-initiated.

The product is a Windows VCL host containing a React and TypeScript editor rendered with WebView2. The editor builds project data and website output locally. Native code provides controlled Windows integration, recovery and experimental SFTP and AI features. The project began as the MIT-licensed OpenPage application; preserve the upstream license, notices and history.

Source tree and ownership

desktop/DelphiWebStudio contains the DPR, DPROJ and design-time DFM and Pascal form units. WebStudio.Main owns the main form, WebView events, native menus and download handling. LocalAIForm, SftpForm and DeploymentReviewForm provide their corresponding design-time dialogs. Form properties belong in the DFM and Object Inspector where possible; MainForm starts maximized through WindowState.

desktop/bridge contains the native request validator and session manager. desktop/project contains project open and save, SFTP connection and inventory, deployment review, preflight, journal and execution, and rollback review and execution. desktop/ai contains local model lifecycle, host and client units. desktop/installer holds the experimental x64 Inno Setup script and notes.

src/main.tsx and src/App.tsx start the editor. src/routes owns dashboard, editor, components, export and settings views. src/editor contains canvas, page and panel controls. src/blocks contains block types and renderers. src/store contains Zustand state. src/project handles portable project files, checks, imports and revisions. src/media implements asset storage and PDF UI. src/ai implements typed site actions and local-plan review. src/desktop contains the browser side of the native bridge. src/lib includes templates, HTML export, link and deployment utilities.

tests contains Vitest tests and native harnesses. docs contains current and historical notes, including PROJECT_FILES, MEDIA_LIBRARY, AI_ACTIONS, import documentation, privacy, recovery and M8_VALIDATION. bin and dcu are generated build areas. docs/guides and docs/pdf hold delivered manuals. test-results holds isolated test artifacts and must not be mistaken for source or release output.

Build tools and configurations

The source specifies React 19, TypeScript 5.9, Vite 7, Zustand, Immer, dnd-kit, JSZip and PDF.js. package-lock.json records exact dependency resolution. package.json defines dev, build, lint, test and preview scripts. Build executes TypeScript project build followed by Vite production build. Tests use Vitest; Vite config supplies the @ alias to src and test include pattern.

The Delphi project is desktop/DelphiWebStudio/DelphiWebStudio.dproj. RAD Studio 37.0 provides rsvars.bat, dcc32 and dcc64. The project declares VCL, both Win32 and Win64 platforms, and architecture-specific bin and dcu output directories. The desktop DPR initializes VCL, creates TMainForm and starts the application. The DFM declares native controls and event bindings; modifying form behavior should preserve IDE editability.

The tested build commands initialize rsvars.bat, then run MSBuild with Config=Release and Platform=Win64 or Win32. The editor build must be staged in an editor subfolder beside the executable, along with the architecture-matched signed WebView2Loader.dll. WebView2 Runtime is a separate prerequisite. These local builds were compiled and startup-window placement was checked; neither clean-machine compatibility nor redistribution rights are established by compilation alone.

The existing Inno Setup installer is explicitly x64 and expects a prebuilt stage. It checks for an installed WebView2 Runtime, does not install Node or the local AI model, and preserves user data on uninstall in the described local tests. It has not been validated as a Win32 installer or on a clean Windows machine. Verify release staging, notices, runtime and edition-specific Delphi terms before distribution.

Entry, routing and startup

The desktop form checks for editor/index.html beside the executable before navigating. TEdgeBrowser maps the local editor directory to the webstudio.local origin; the browser app selects HashRouter when location.origin is https://webstudio.local and BrowserRouter otherwise. Routes include dashboard, editor, components, deploy and settings. Native and browser routing must be evaluated together if the virtual origin or base path changes.

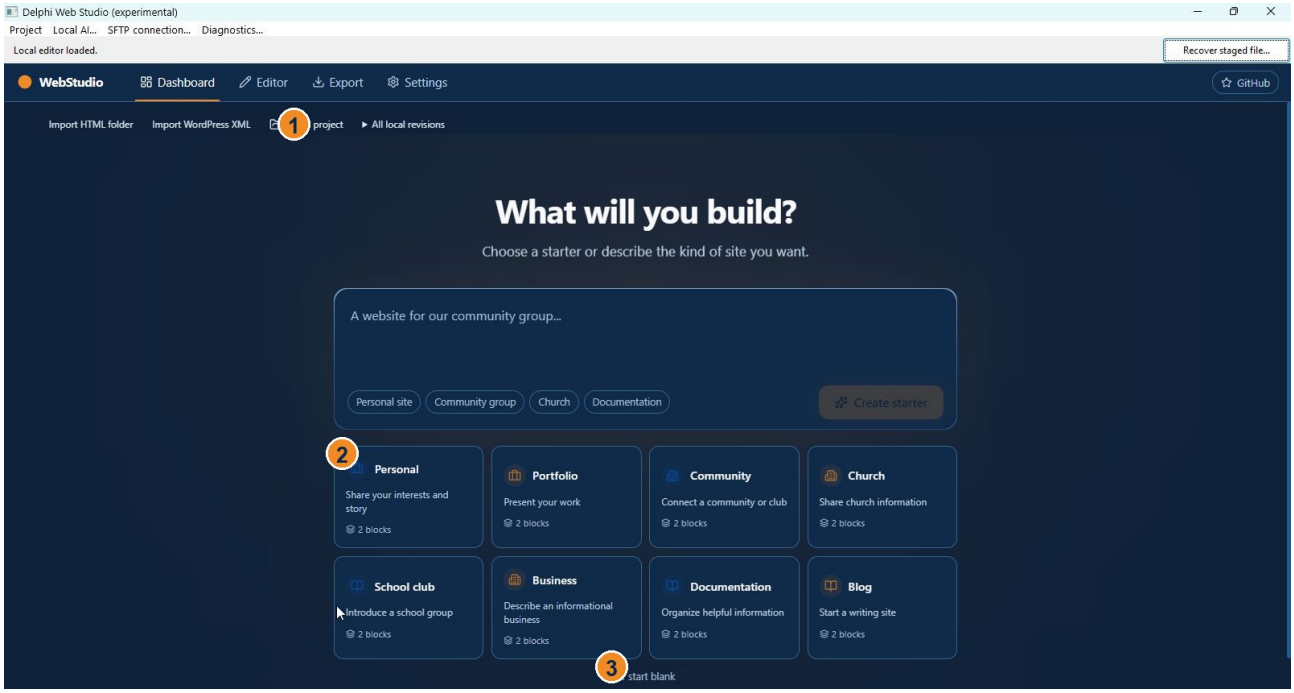


Figure 2. Current local desktop host displaying its dashboard.

The host registers WebView creation, navigation, frame navigation, navigation-complete, new-window, permission, download and web-message callbacks. Its origin and permission checks are part of the trust boundary. Do not loosen the local origin, navigation or message validation merely to make a feature work; test untrusted sources and any affected desktop workflow first. The blocked custom-code frame limits desktop preview even though exported content may execute in a visitor's browser.

Project schema and validation

src/project/project-file.ts controls the portable-project feature flag, schema-2 parsing, legacy migration, serialization and validation. VITE_WEBSTUDIO_PROJECTS is enabled unless explicitly set to false. A project file carries a format marker, schemaVersion, projectId and config; it can retain unknown fields for forward compatibility. Existing OpenPage single-page JSON is migrated in memory and opened as a separate draft rather than overwritten in place.

The validator checks core object structures, identities, paths, duplicate page and block identifiers, unsupported format versions, URL protocols, known theme values and unsafe keys. Unknown block types are preserved as project data and shown as placeholders. Export refuses unsupported blocks rather than silently discarding them. The validation contract does not imply every semantic property or malicious archive combination has been exhaustively analyzed.

Page state, editor state and project settings are distributed between Zustand stores. configStore tracks site configuration, active page and undo/redo snapshots. projectsStore tracks projects, project config and settings. editorStore holds UI session state such as selection, preview and session deployment credentials. Page identifiers and relative addresses are important to navigation and export; migrations must preserve existing IDs and content or reject conflicting input.

Persistence, media and revisions

Draft data uses localStorage keys openpage-projects and openpage-config with checks designed to preserve unreadable or incompatible stored values rather than overwrite them. The WebView profile is local user data. A new feature must not clear or reset that profile as a routine startup or uninstall operation.

ProjectMediaLibrary uses IndexedDB database webstudio-media, version 3, with the assetsV2 store keyed by projectId and id. Imported assets are project-scoped and retain metadata including size and SHA-256. The import pipeline validates supported types, handles duplicate decisions and stages content before committing project references. Media originals supplied by the user are not intentionally altered. Project save and website export perform integrity checks on relevant data.

ProjectRevisions uses the webstudio-project-revisions IndexedDB database and revisions store. Local revisions are separate from portable project files. New revisions record checksums; older ones can lack them. Recovery opens an independent copy, and saving a revision as a project file is a distinct operation. Tests exercise selected transaction-abort and forced-termination scenarios, not every interruption or power-loss case.

ProjectFileCommands coordinates browser-side open, save, import, revision and recovery UI. A project with media can be packaged as a ZIP containing manifest and assets; a media-free project can be JSON. Staging or cancellation can leave local remnants after interruption. Keep profile data, project files and backups distinct when diagnosing failures.

Native save, backup and recovery

WebStudio.ProjectOpen handles bounded native reads. WebStudio.ProjectSave implements staged output, verified project backup ZIP creation and restoration, and guarded replacement. The desktop download handler routes recognized project output through native staging and confirmation. New destinations are created without intentionally replacing another file. Existing-target replacement requires explicit confirmation and a checked recovery ZIP.

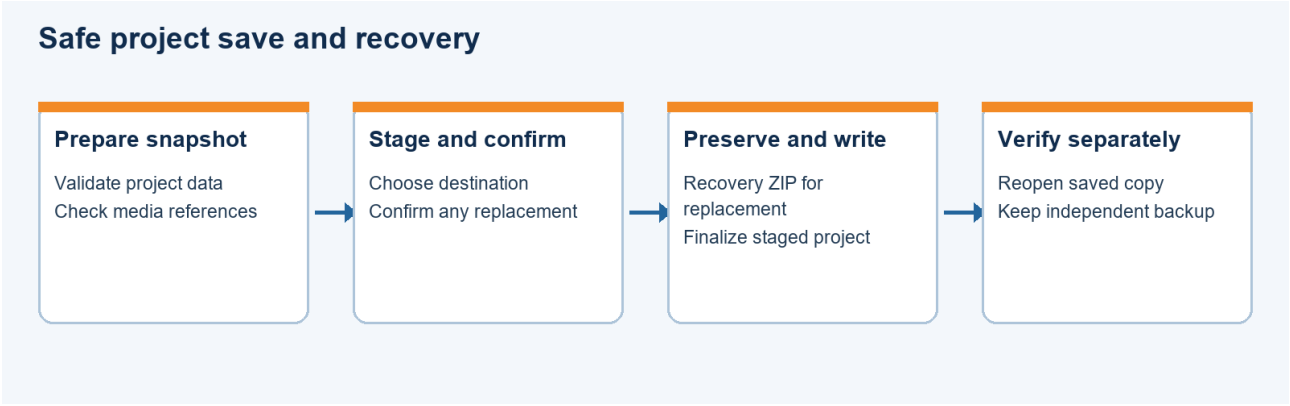


Figure 3. The documented staged project-save and independent verification sequence.

The replacement path validates the stage, checks file identities and attributes, creates recovery material and invokes Windows replacement APIs. It reports paths needed for recovery when operations fail. This is not a proof against all hostile directory changes, competing writers, device failures or power loss. Reopen saved projects and preserve independent backups. Backup and restore limits documented in the current UI are 512 MiB for a project and 513 MiB for a recovery archive.

The native Project menu offers backup and restore. Recover staged file recognizes a retained temporary JSON or ZIP and moves it to a new name in the same folder; format recognition is preliminary and does not prove a complete project. Shutdown and download callbacks must be handled carefully to avoid losing status or destroying active WebView event handlers.

Native bridge and trust boundary

WebStudio.Bridge defines dcProjectOpen, dcProjectSave, dcMediaImport and dcSiteExport capabilities. ValidateDesktopRequest accepts a version-1 JSON envelope with exactly five fields and a maximum UTF-8 size of 16384 bytes. It checks the sender URL prefix, request ID syntax, native session ID, project ID and granted capability. Non-open operations require an active project. It does not itself perform a file operation.

TDesktopSession generates a 32-byte random session identifier through BCryptGenRandom, tracks claimed request IDs under a monitor, rejects replays and limits a session to 4096 requests. Revocation clears grants. Sender URL must come from the WebView event, and grants must be determined by the host rather than trusted from page data. The JavaScript protocol and native handler must evolve together; adding a capability requires explicit review, validation and appropriate tests.

Rendering, controls and export

The editor renders React blocks on a canvas. PropertiesPanel declares editable fields for supported block types; DesignPanel applies theme tokens and the toolbar controls pages, preview widths, history and export. Block creation and selection use the registry and metadata. The current architecture has separate preview and static-export renderers, so adding or changing a block must update both paths or clearly reject export.

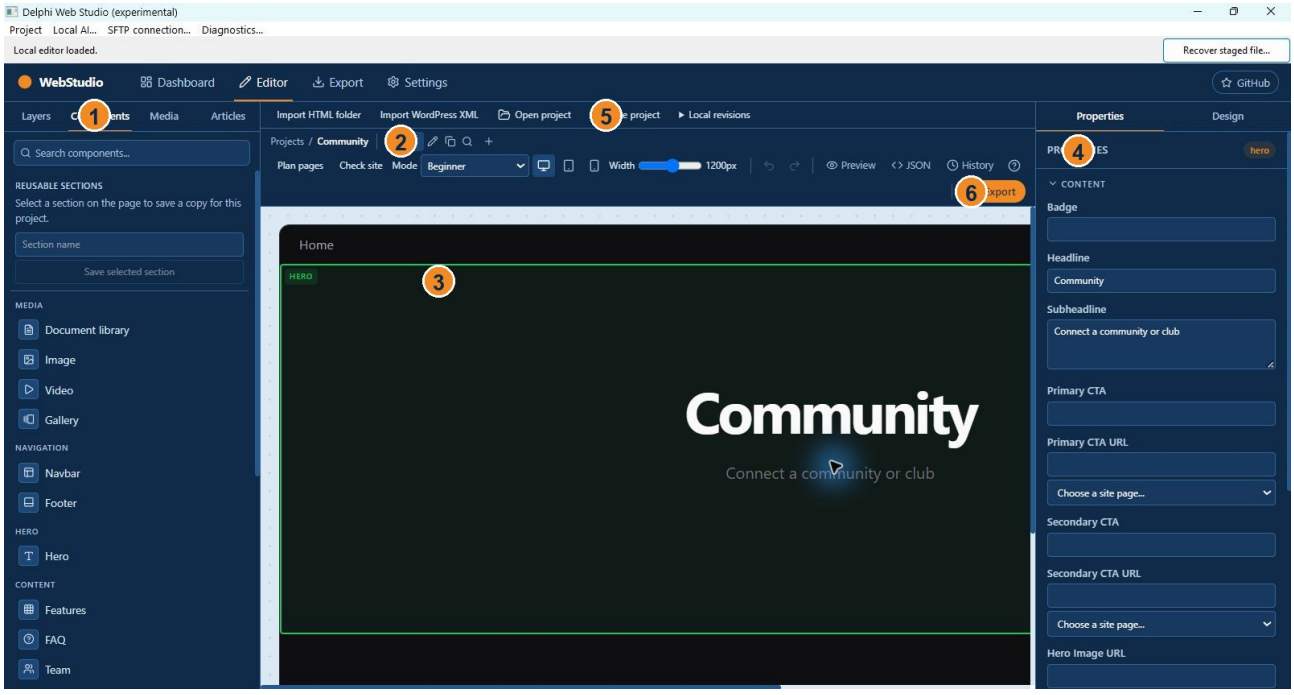


Figure 4. Editor workspace with component palette, page canvas and native host.

src/lib/export-html.ts exports site rendering and exportSiteZip. The ZIP path creates separate HTML pages and supported local assets, includes PDF.js resources when needed, adjusts page and asset references, and applies

project metadata. External URLs supplied by users can still cause network use. User-supplied custom code can execute in the exported site and must be treated as untrusted content.

Check site validates selected known references, metadata, image descriptions, form configuration and media. It is not a full link crawler, browser compatibility test or accessibility audit. Test generated sites on intended browsers and hosting paths, including nested paths, and distinguish a local ZIP from an actual deployment.

Static and WordPress import

static-html-import.ts converts selected static HTML and CSS into a separate project. Its UI applies documented caps for file count, page count, combined text and image data. Basic recognized structures and local references can become editable content. Original text and reports are retained as inactive project information. Unsupported markup can be retained as an inert read-only block that is excluded from website export. It is not a general browser-grade visual reconstruction engine.

wordpress-import.ts reads supported WXR XML with limits, rejects document-type and entity declarations, and converts supported posts and pages to project content. Posts and unpublished pages begin as drafts. Attachments, plugins, themes, shortcodes, unmatched links and server behavior are not generally recreated. Retained XML may contain private author information and is included in project data; sharing and privacy decisions must account for it.

Local AI and optional cloud AI

site-actions.ts defines a limited typed action vocabulary, preview and acceptance. A plan is checked against the current project snapshot and requires user approval; acceptance is undoable. Supported local flows include five-page planning for a blank project, a single page and limited color suggestions. Generated text is untrusted and can invent facts. There is no demonstrated general conversational editor or complete quality benchmark.

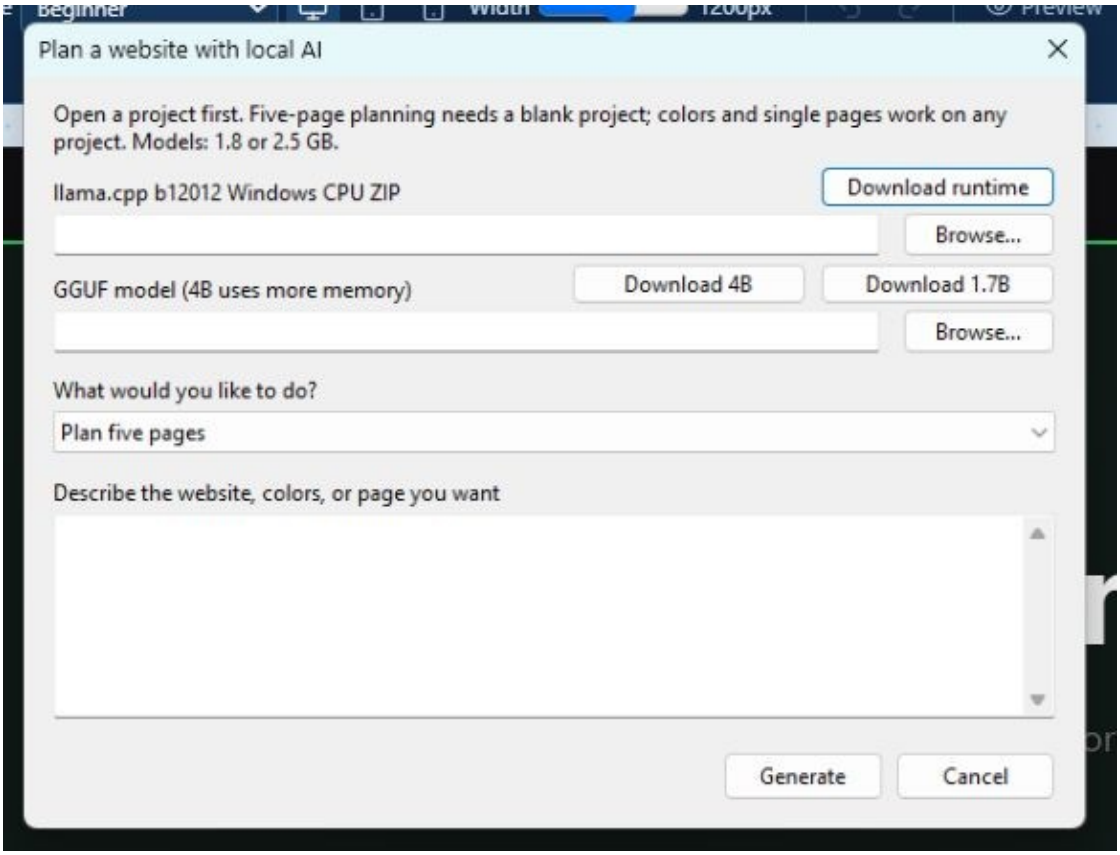


Figure 5. Native Local AI dialog; files and a proposal must be explicitly selected.

Native local AI uses the configured supported runtime ZIP and GGUF model. The implementation verifies pinned checksums, starts a supervised local process on loopback with a temporary key and performs ownership/readiness checks before requests. The offered Windows CPU runtime is x64, including when the editor host is Win32. Runtime and models are downloaded separately, not included in the current editor folders. Reassess exact license, provenance and distribution requirements before packaging.

The cloud-generation path is separate and disabled until session opt-in. New Gemini and deployment keys are held in memory for the session; legacy keys may remain in local storage or settings until removed. Key testing contacts Google. Treat generated requests, third-party endpoints and logs as sensitive. Avoid sending private content without user authorization.

SFTP review, transfer and rollback

WebStudio.SftpConnection invokes the Windows OpenSSH sftp.exe with a user-selected private key and known-hosts file. TestSftpConnection does not intentionally upload, remove or modify remote files. Inventory and backup are separate steps with documented size limitations; links are not supported by the UI workflow. Connection identity, endpoint configuration and private recovery-folder protection remain user and host responsibilities.

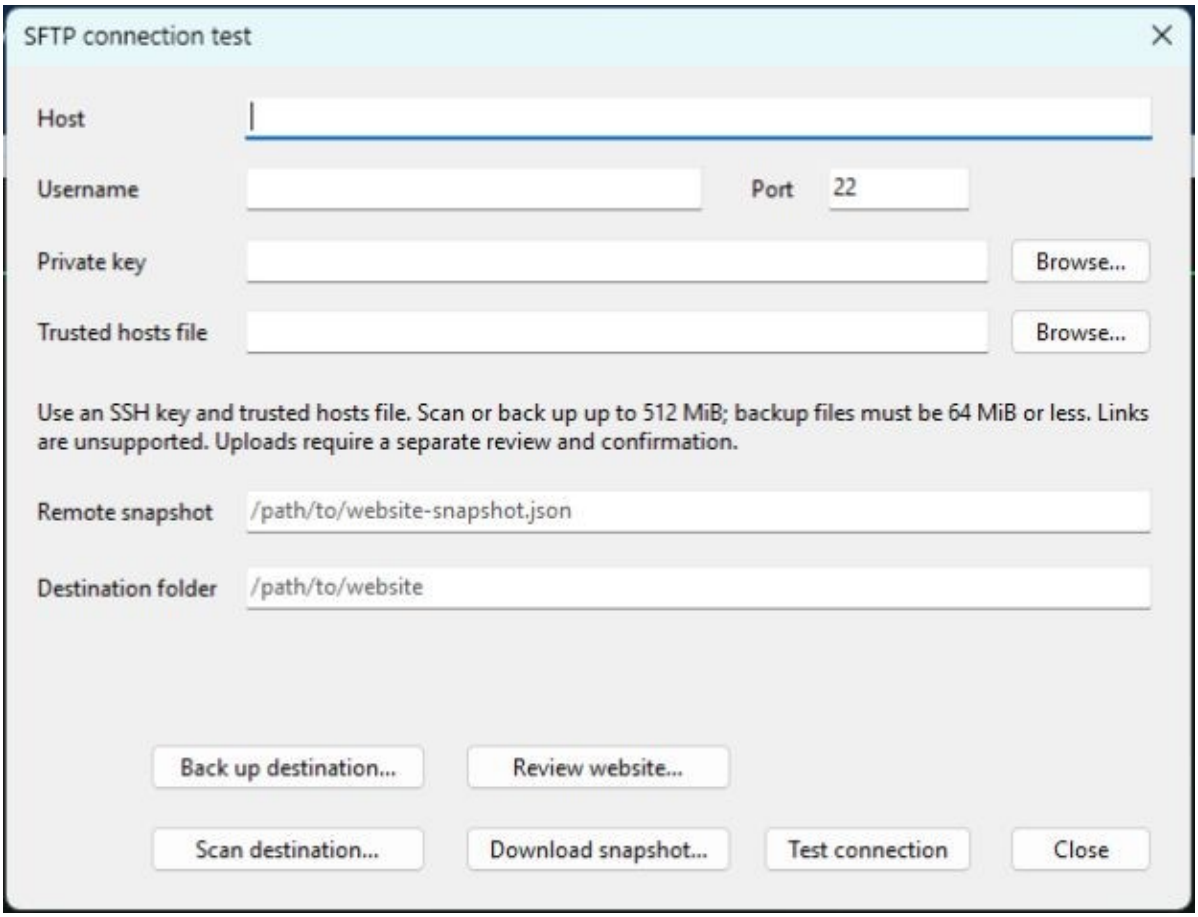


Figure 6. The native SFTP dialog; upload remains a separate confirmed action.

DeploymentReview, Preflight, Journal and Executor bind reviewed website ZIP, destination scan and backup to checksums and preflight conditions. ExecuteReviewedDeployment requires Confirmed=True; the caller must obtain explicit user approval and ensure the recovery directory is not publicly served. Destination-only files are retained. Transfer receipts and recovery material help diagnose failures but a partially completed transfer can change the destination.

RollbackReview and RollbackExecutor support reviewed recovery scenarios. Synthetic loopback tests do not establish compatibility with every actual host, race, SSH configuration or rollback failure. Never enable or perform a real publish as part of a documentation or test build without the user's separate explicit instruction.

Validation and known issues

The last recorded M8 run passed 273 tests across 19 files; two default integration tests were skipped, one of which was separately run offline. Production editor build, TypeScript and lint passed in the recorded environment. A current-source isolated test selected HTML and WordPress imports through real Windows dialogs. A five-page project and PDF passed documented local round trips. These are scoped observations, not complete product acceptance.

The recorded npm audit after compatible updates had 22 development-dependency findings, including four high-severity entries; npm audit --omit=dev reported zero at that time. The optional Vercel toolchain includes pinned transitive versions, so forced overrides or breaking upgrades require separate assessment and testing.

Audit data changes over time. Native Delphi redistribution, third-party notices, optional model terms, clean-Windows installation, full accessibility and real-host publication remain open.

The Windows x86 and x64 source builds and maximized window placement were locally checked. The older experimental installer is x64 only. The x64 AI runtime is not an x86 runtime. Neither architecture has a clean-Windows end-to-end test in the evidence cited here. Treat earlier historical documents describing absent features as baseline observations, not current behavior.

Maintenance checklist

Before a change, confirm branch and user instructions, identify exact source and behavior, and create a verified pre-change ZIP at the authorized backup location. Preserve user project files and unrelated untracked data. Change form properties at design time, update both preview and export as needed, and keep bridge capabilities constrained. Build appropriate target architectures and run focused, meaningful tests.

For a release candidate, separately verify license and edition obligations; stage editor assets and matching loaders; test clean Windows installation; open, save, reopen and export a realistic project; test accessibility and supported devices; validate imports and recovery; inspect dependencies; and record any remaining limitations. Never equate compilation or one sample with release readiness. Commit only intended validated files and push approved backup remotes without force.

Main form lifecycle in detail

TMainForm.FormCreate resolves the editor directory relative to ParamStr(0). If editor/index.html is absent it reports that files must be staged and does not initialize the WebView. Otherwise CreateWebView begins asynchronous creation; an exception becomes a status message. This means a compiled EXE alone is insufficient to start the editor.

EditorBrowserCreateWebViewCompleted checks the result and ICoreWebView2_3 interface, enables web messages, maps the editor directory to webstudio.local using deny access behavior, and navigates to the local index. Navigation completion updates the status. Top-level navigation starting revokes and frees an existing bridge session and rejects addresses outside the local prefix. Frame navigation is cancelled, new windows are handled, and permission requests are processed by the native host. Those details affect local HTML previews and external links; consult the event code before changing them.

FormCloseQuery blocks closing while local AI is working. During an active save it displays a warning and initially prevents closing; another close can interrupt the operation. FormDestroy revokes and frees a bridge session. Download callbacks and queued work have their own lifetime concerns. Changing shutdown handling requires testing with an in-progress save and the application's own isolated test process, preserving user data.

LocalAIItemClick requires an initialized WebView and an open editor URL. The form collects a runtime archive, model, brief and proposal type. The host creates a request ID, informs the web editor that planning started, and runs inference in a background anonymous thread. It queues completion or failure to the main thread and dispatches a local-plan event to the editor. The source retains an FLocalAIWorking flag and disables the menu during processing. Do not equate a queued event with an accepted change: review and acceptance remain editor-side operations.

Browser and native request sequence

requestDesktopAction in src/desktop/native-bridge.ts is an authorization step for an existing local flow; its documented request carries no file path or project content. Outside the mapped local host it returns without using the native bridge. In the desktop it validates the project ID and obtains chrome.webview. A request waits at most five seconds per exchange and removes the message listener on completion or timeout.

The first exchange sends a version-1 session.open message containing requestId, projectId and requested capability. The native host validates the source and envelope, discards any prior session and creates a session granted the one requested capability. The response returns a 64-character hexadecimal session ID and echoes request identity and capability. The second exchange sends the five-field request using a new ID and the returned session ID. The host validates and claims that request and responds with ok. Unexpected reply identities or missing fields are rejected.

The Pascal and TypeScript validators are complementary but not identical implementations: the TypeScript side parses senderUrl using URL and tests its origin; the Pascal side accepts an event-supplied local URL prefix and rejects control characters and backslashes in the tail. The native host's event Source remains authoritative. Both sides constrain request IDs and capabilities. Test malformed input, source confusion, replay, project mismatch, session revocation and timeout when changing either implementation.

Media record and import mechanics

MediaAsset records include projectId, id, name, path, MIME, byte size, SHA-256 and content, with optional import time, folder and dimensions. Replacement can retain one previous version and a previous name. The IndexedDB compound primary key separates identical IDs in different projects; indexes support project enumeration and project-plus-hash lookup. Database onversionchange closes the connection, and blocked opening reports an error instead of silently deleting data.

The import method maps extensions to expected MIME types, checks representative byte signatures, uses a restricted element and attribute policy for SVG, and attempts image decoding where available. PDF import asks PDF.js to parse the file, with special handling for a password-required response. The complete file is read into a byte array for these operations and hashing, so large files and memory usage require practical assessment. Signature checks and SVG restrictions are defense layers, not an assertion that every parser or downstream consumer is immune to malformed content.

Restored assets must satisfy identity, path and hash checks. New assets use UUIDs and sanitized filename fragments in their generated assets path. Folder paths reject dot and parent components. Duplicate checking and insertion use an IndexedDB transaction; project copying uses projectId-indexed cursors. Replacement compares the current record with the observed original before writing and retains the previous data for one-level undo. Any extension to the asset schema must preserve the transaction, migration and portable-archive behavior.

Project file and archive maintenance

decodeProjectFile performs strict UTF-8 decoding. parseProjectFile accepts the supported portable format or migrates an older configuration, then validates. serializeProjectFile produces the portable representation after validation. validateProjectMedia checks consistency of references and asset records. Project import and save

flows are asynchronous and compare the active project or media state where implemented; a state change can cause the operation to stop or warn rather than silently replace newer work.

ZIP handling uses JSZip on the browser side and System.Zip in native backup and deployment paths. A project manifest and an exported site ZIP are different artifacts with different validation and privacy implications. Unknown document, config, page and block properties should round-trip where supported. A new schema version needs an explicit migration plan, compatibility tests, safe handling for newer versions and preservation of the original source.

The supported static-import report retains source information and issue summaries; unsupported original markup is not executed by the source inspector or the retained inert source block. The export intentionally excludes that block. Modifying the import/export contract requires tests that check both preservation of raw source in a project and its absence from executable website output.

Testing map and failure interpretation

Vitest discovers tests/**/*.test.ts through vite.config.ts. The project-file, project-storage, project-revision, page-history, media, AI-action, HTML and multi-page export tests cover important business rules. The exact current test files and scripts should be inspected when selecting a targeted run; the 273-pass figure is a historical recorded result and must not be presented as a fresh result after later source changes.

Native harnesses in tests/native exercise selected desktop code and WebView flows. DesktopFivePageRoundtripSmoke and DesktopModernFivePageHarness cover the documented project round trip with different dialog modes; separate tests cover project save, recovery, desktop bridge and local AI. Their fixtures, test profile, Windows runtime and dialog choice are part of the evidence. An isolated test should write only disposable data and should not use a user's live project or publish to a live destination.

When a build reports success, distinguish compilation from staging and startup. When an export passes a sample, distinguish the sample's content and viewport from every possible block. When a recovery test passes, state its exact interruption point. When an accessibility scanner passes, record its page, rules and viewport. The current records do not prove clean-Windows operation, full host compatibility or freedom from data loss.

Delivery, architecture and platform limitations

The Win32 and Win64 project configurations emit to separate bin and dcu directories. WebView2Loader.dll must match the process architecture; editor assets belong in the sibling editor folder. The desktop is VCL, even though the original handoff proposed FMX. The local AI download button points to a Windows x64 runtime archive; a Win32 host does not make that external runtime compatible with 32-bit Windows.

The current installer notes describe only an experimental per-user Win64 package and predate the current dual-architecture desktop build. Do not describe the Win32 executable as a tested 32-bit installer. On a real target, verify WebView2 Runtime availability and any Windows OpenSSH dependency for SFTP, and confirm that local AI files are separately obtained. Runtime dependencies, applicable licenses and release packaging must be reviewed together.

The project includes both older baseline documentation and later implementation evidence. When they disagree, inspect source and the most recent targeted validation; do not copy historical limitations as current

facts or assume later notes guarantee broader behavior. Record the source revision and date of any future guide update so maintainers can assess drift.

Source declaration index

The following index is generated from the checked-in project source. It lists selected public TypeScript exports and Pascal interface declarations to help navigate the implementation; signatures are source references, not claims that every declared operation is connected to every user flow. Read the corresponding implementation and tests before making changes.

desktop/bridge/WebStudio.Bridge.pas

Declared types or operations: TDesktopCapability, TDesktopRequest, TDesktopSession, Create, Destroy, ValidateAndClaim, Revoke, ValidateDesktopRequest.

desktop/project/WebStudio.DeploymentExecutor.pas

Declared types or operations: TDeploymentExecution, ExecuteReviewedDeployment.

desktop/project/WebStudio.DeploymentJournal.pas

Declared types or operations: TDeploymentJournal, WriteRecord, CreateNew, Destroy, RecordDirectoryIntent, BeginTransfer, CompleteTransfer, FailTransfer, Stop, RecordFinalVerification, Finish.

desktop/project/WebStudio.DeploymentPreflight.pas

Declared types or operations: CheckDeploymentPreflight.

desktop/project/WebStudio.DeploymentReview.pas

Declared types or operations: TDeploymentTransfer, TDeploymentReview, ReviewDeployment.

desktop/project/WebStudio.ProjectOpen.pas

Declared types or operations: ReadProjectFile.

desktop/project/WebStudio.ProjectSave.pas

Declared types or operations: SaveNewProjectFile, CommitStagedProjectFile, ReplaceStagedProjectFile, SaveProjectRecoveryZip, RestoreProjectRecoveryZip.

desktop/project/WebStudio.RollbackExecutor.pas

Declared types or operations: ExecuteAddedRollback.

desktop/project/WebStudio.RollbackReview.pas

Declared types or operations: ReviewAddedRollback.

desktop/project/WebStudio.SftpConnection.pas

Declared types or operations: TestSftpConnection, DownloadSftpSnapshot.

desktop/project/WebStudio.SftpInventory.pas

Declared types or operations: TSftpInventoryEntry, TSftpTransferReceipt, ScanSftpInventory, SaveSftpInventory, BackupSftpDestination, UploadNewSftpFile, UploadNewSftpFileWithId, ReplaceSftpFile, ReplaceSftpFileWithId, TransferSftpStreamWithId, RestoreSftpOriginal, RetainAddedSftpFile,

RetainAddedSftpFileWithId, EnsureSftpDirectories, NormalizePrivateRecoveryDirectory, CheckPrivateSftpRecovery.

desktop/ai/WebStudio.LocalAIClient.pas

Declared types or operations: GenerateLocalSitePlan.

desktop/ai/WebStudio.LocalAIHost.pas

Declared types or operations: RunLocalSitePlan.

desktop/ai/WebStudio.LocalAIProcess.pas

Declared types or operations: TLocalAIProcess, Destroy, Start, Stop, IsRunning, ProcessId.

desktop/DelphiWebStudio/WebStudio.DeploymentReviewForm.pas

Declared types or operations: TDeploymentReviewForm, WebsiteBrowseClick, ScanBrowseClick, BackupBrowseClick, ReviewButtonClick, InputChanged, CheckButtonClick, PublishButtonClick, ReceiptBrowseClick, RollbackReviewButtonClick, RollbackButtonClick, FormCloseQuery, SetConnection.

desktop/DelphiWebStudio/WebStudio.LocalAIForm.pas

Declared types or operations: TLocalAIForm, RuntimeBrowseButtonClick, ModelBrowseButtonClick, RuntimeDownloadButtonClick, ModelDownloadButtonClick, LargeModelDownloadButtonClick, GenerateButtonClick.

desktop/DelphiWebStudio/WebStudio.Main.pas

Declared types or operations: TMainForm, FormCreate, FormDestroy, FormCloseQuery, EditorBrowserCreateWebViewCompleted, EditorBrowserNavigationStarting, EditorBrowserFrameNavigationStarting, EditorBrowserNavigationCompleted, EditorBrowserNewWindowRequested, EditorBrowserPermissionRequested, EditorBrowserDownloadStarting, EditorBrowserWebMessageReceived, RecoverStageButtonClick, BackupProjectItemClick, RestoreBackupItemClick, LocalAIItemClick, SftpItemClick, DiagnosticsItemClick.

desktop/DelphiWebStudio/WebStudio.SftpForm.pas

Declared types or operations: TSftpForm, KeyBrowseClick, HostsBrowseClick, TestButtonClick, FormCloseQuery, DownloadButtonClick, ScanButtonClick, BackupButtonClick, ReviewButtonClick.

src/project/project-file.ts

Declared types or operations: portableProjectsEnabled, SiteProjectFile, decodeProjectFile, readProjectZipManifest, validateProjectConfig, createProjectFile, validateProjectMedia, parseProjectFile, serializeProjectFile.

src/project/project-revisions.ts

Declared types or operations: ProjectRevision, ProjectRevisions.

src/project/site-checks.ts

Declared types or operations: SiteIssue, checkSite.

src/project/static-html-import.ts

Declared types or operations: importStaticHtml.

src/project/wordpress-import.ts

Declared types or operations: importWordPressXml.

src/media/media-library.ts

Declared types or operations: MediaAsset, ProjectMediaLibrary.

src/ai/site-actions.ts

Declared types or operations: SiteAction, SiteActionRequest, SiteActionPreview, previewFivePagePlan, previewSinglePagePlan, previewSiteActions, acceptSiteActions.

src/desktop/bridge-protocol.ts

Declared types or operations: DesktopCapability, DesktopRequest, DesktopSession, validateDesktopRequest.

src/desktop/native-bridge.ts

Declared types or operations: requestDesktopAction.

src/lib/export-html.ts

Declared types or operations: ExportSiteSettings, ExportSiteOptions, exportSiteToHTML, exportToHTML, exportSiteZip, downloadHTML, previewHTML.